T Bhanu Teja

192472259

C++ PROGRAMMING

UNIT IV – INHERITANCE & POINTERS

QUESTIONS WITH ANSWERS

Assessment Coverage

Section A: 5 debugging problems with defect explanation, corrected C++ programs, dry runs and outputs.

Section B: 5 structured pseudocode problems with design notes, execution traces and expected results.

Topics include hybrid and virtual inheritance, constructor delegation, base pointers, object slicing, this-pointer chaining, access control, abstract classes, composition, multilevel inheritance and pointer-based sorting.

SECTION A – DEBUGGING QUESTIONS

Question 1: The Diamond Problem in Multipath Hybrid Inheritance

Topic: Hybrid Inheritance, Virtual Base Classes, Ambiguity

Problem Statement: Student and Employee both inherit Person. TeachingAssistant inherits both, creating two Person subobjects and making idNumber ambiguous.

Defect category and location: Ambiguous member access at the statement ta.idNumber. Student and Employee use ordinary (non-virtual) public inheritance.

Explanation: The TeachingAssistant object contains one Person subobject through Student and another through Employee. Therefore, the compiler cannot decide which idNumber is requested. Virtual inheritance makes Student and Employee share one Person base subobject. The most-derived class, TeachingAssistant, initializes that virtual base.

Corrected C++ Code:

```
#include <iostream>
using namespace std;

class Person {
public:
    int idNumber;
    Person(int id = 0) : idNumber(id) {
        cout << "Person initialized: " << idNumber << endl;
    }
};

class Student : virtual public Person {
public:
    Student(int id) : Person(id) {}
};

class Employee : virtual public Person {
public:
    Employee(int id) : Person(id) {}
};

class TeachingAssistant : public Student, public Employee {
public:
    TeachingAssistant(int id)
        : Person(id), Student(id), Employee(id) {}
};

int main() {
    TeachingAssistant ta(101);
    cout << "TA ID: " << ta.idNumber << endl;
```

```
    return 0;
}
```

Dry Run: TeachingAssistant(101) constructs its single virtual Person base first. Student and Employee are then constructed; their Person initializers are ignored for the virtual base because TeachingAssistant is the most-derived class. Finally, the main body accesses the single idNumber.

Step	Operation	Result
1	Construct virtual Person(101)	Person initialized: 101
2	Construct Student(101)	No second Person object
3	Construct Employee(101)	No second Person object
4	Access ta.idNumber	101

Expected Output:

```
Person initialized: 101
TA ID: 101
```

Note: The source asks for the exact compiler error. Diagnostic wording varies by compiler; the essential error is that the request for member idNumber is ambiguous.

Question 2: Missing Base Constructor in Derived Initializer List

Topic: Constructors in Derived Classes, Constructor Execution Order

Problem Statement: Device has only a parameterized constructor. Sensor does not call it in its initializer list, so the compiler attempts to call a nonexistent Device() before entering the Sensor constructor body.

Defect category and location: Missing base-class constructor invocation in Sensor(std::string mac, int rate). Assigning macAddress inside the body is too late; base construction happens first.

Corrected C++ Code:

```
#include <iostream>
#include <string>
using namespace std;

class Device {
protected:
    string macAddress;
public:
    Device(string mac) : macAddress(mac) {
        cout << "Device configured with MAC: "
             << macAddress << endl;
```

```
    }
};

class Sensor : public Device {
private:
    int samplingRate;
public:
    Sensor(string mac, int rate)
        : Device(mac), samplingRate(rate) {
        cout << "Sensor initialized." << endl;
    }

    void display() const {
        cout << "MAC: " << macAddress
             << ", Rate: " << samplingRate
             << " Hz" << endl;
    }
};

int main() {
    Sensor tempSensor("00:1A:2B:3C:4D:5E", 100);
    tempSensor.display();
    return 0;
}
```

Dry Run: The Device base constructor executes before the Sensor constructor body. After the base is ready, samplingRate is initialized and the Sensor body runs.

Order	**Initialization / Action**	**Result**
1	Device(mac)	MAC address stored and configuration message printed
2	samplingRate(rate)	100 stored
3	Sensor constructor body	Sensor initialized message printed
4	display()	MAC and rate displayed

Expected Output:

```
Device configured with MAC: 00:1A:2B:3C:4D:5E
Sensor initialized.
MAC: 00:1A:2B:3C:4D:5E, Rate: 100 Hz
```

Question 3: Base Class Pointer Member Access & Object Slicing

Topic: Pointers to Derived Classes, Static Type Checking, Object Slicing

Problem Statement: Vehicle* can point to a SportsCar object, but the pointer type only exposes Vehicle members. Copying SportsCar into a Vehicle object also slices away the derived portion.

Defect category and location: vPtr->engageTurbo() is invalid because engageTurbo is not declared in Vehicle. Vehicle plainVehicle = car; causes object slicing: turboEnabled is not copied into plainVehicle.

Corrected C++ Code:

```
#include <iostream>
using namespace std;

class Vehicle {
public:
    int maxSpeed;
    Vehicle(int s) : maxSpeed(s) {}
    void honk() {
        cout << "General Horn" << endl;
    }
};

class SportsCar : public Vehicle {
public:
    bool turboEnabled;
    SportsCar(int s, bool turbo)
        : Vehicle(s), turboEnabled(turbo) {}

    void engageTurbo() {
        cout << "Turbo Engaged!" << endl;
    }
};

int main() {
    Vehicle* vPtr = new SportsCar(300, true);
    vPtr->honk();

    SportsCar* sportsPtr = dynamic_cast<SportsCar*>(vPtr);
    if (sportsPtr != nullptr) {
        sportsPtr->engageTurbo();
    }

    SportsCar car(250, true);
    Vehicle& vehicleRef = car;  // No slicing
    cout << "Speed through base reference: "
```

```
        << vehicleRef.maxSpeed << endl;
    cout << "Turbo enabled: "
        << (car.turboEnabled ? "Yes" : "No") << endl;

    delete vPtr;
    return 0;
}
```

Dry Run: The base pointer points to a SportsCar allocated on the heap. It can call honk(), because honk belongs to Vehicle. dynamic_cast checks the actual object type and safely obtains a SportsCar pointer. A base reference refers to the original object and does not create a sliced copy.

Step	Statement	Result
1	new SportsCar(300, true)	Vehicle part: 300; turboEnabled: true
2	vPtr->honk()	General Horn
3	dynamic_cast<SportsCar*>(vPtr)	Successful downcast
4	sportsPtr->engageTurbo()	Turbo Engaged!
5	Vehicle& vehicleRef = car	Reference to existing SportsCar; no slicing

Expected Output:

```
General Horn
Turbo Engaged!
Speed through base reference: 250
Turbo enabled: Yes
```

Important: dynamic_cast requires a polymorphic base class (at least one virtual function). The supplied Vehicle class has no virtual function, so to keep this corrected program fully compilable with dynamic_cast, add a virtual destructor to Vehicle. The code below includes that requirement:

```
virtual ~Vehicle() = default;
```

Place this declaration in Vehicle public section. It also makes deletion through a base pointer safe when derived classes later own resources.

Question 4: Incorrect Return Type in this Pointer Method Chaining

Topic: this Pointer, Method Chaining (Fluent Interface)

Problem Statement: setWidth returns WindowBox*, but the next chained call uses the dot operator. A pointer requires ->, while fluent chaining with dot syntax should return WindowBox& and return *this.

Defect category and location: Return type WindowBox* and return this in setWidth().

Corrected C++ Code:

```
#include <iostream>
using namespace std;

class WindowBox {
private:
    int width;
    int height;
public:
    WindowBox() : width(0), height(0) {}

    WindowBox& setWidth(int width) {
        this->width = width;
        return *this;
    }

    WindowBox& setHeight(int height) {
        this->height = height;
        return *this;
    }

    void display() const {
        cout << "Window: " << width
             << " x " << height << endl;
    }
};

int main() {
    WindowBox box;
    box.setWidth(800).setHeight(600);
    box.display();
    return 0;
}
```

Dry Run: setWidth stores 800 and returns a reference to the same object. setHeight is then called on that object, stores 600 and returns the same reference. display prints both values.

Step	Operation	Object State
1	Initial object	width=0, height=0
2	setWidth(800)	width=800, height=0
3	setHeight(600)	width=800, height=600
4	display()	Window: 800 x 600

Expected Output:

```
Window: 800 x 600
```

Difference: this is a pointer to the current object (WindowBox*); *this is the current object as an lvalue, so it can be returned as WindowBox&.

Question 5: Private Inheritance Access Violations & Inaccessible Base Cast

Topic: Access Specifiers in Inheritance, Private Derivation, Upcasting

Problem Statement: SecureChannel privately inherits RawSocket. Public base members become private members of SecureChannel, so client code cannot call closePort() or convert SecureChannel* to RawSocket*.

Defect category and location: Private inheritance declaration and inaccessible operations in main(). openPort() remains callable inside SecureChannel, but closePort() is not publicly exposed.

Corrected C++ Code – Approach A (using declaration):

```
#include <iostream>
using namespace std;

class RawSocket {
public:
    void openPort(int port) {
        cout << "Port " << port << " opened." << endl;
    }
    void closePort() {
        cout << "Port closed." << endl;
    }
};

class SecureChannel : private RawSocket {
public:
    using RawSocket::closePort;

    void establishEncryptedTunnel(int port) {
        openPort(port);
        cout << "Tunnel encrypted with TLS 1.3" << endl;
    }
};

int main() {
    SecureChannel channel;
    channel.establishEncryptedTunnel(443);
    channel.closePort();
    return 0;
```

```
}
```

Dry Run: SecureChannel uses openPort internally. The using declaration makes closePort publicly accessible through SecureChannel. The program does not perform an upcast; private inheritance still prevents public conversion to RawSocket*.

Step	Operation	Result
1	establishEncryptedTunnel(443)	Port 443 opened.
2	Print tunnel message	Tunnel encrypted with TLS 1.3
3	channel.closePort()	Port closed.

Expected Output:

```
Port 443 opened.
Tunnel encrypted with TLS 1.3
Port closed.
```

Alternative: Public inheritance permits public upcasting and exposes public base methods. Composition is another design option when SecureChannel has-a RawSocket rather than being-a RawSocket.

SECTION B – PSEUDO-CODE WRITING QUESTIONS

The following solutions use structured pseudocode and retain the class names, attributes and operations specified in the assessment.

Question 1: University ERP Hybrid Hierarchy with Virtual Base Classes

Topic: Hybrid Inheritance, Virtual Base Classes, Diamond Resolution

Scenario: Person is the common root. Student and Faculty virtually inherit Person. TeachingAssistant inherits both and adds stipendAmount and assignedCourse.

Design Overview: Virtual inheritance ensures TeachingAssistant contains one shared Person subobject. Since TeachingAssistant is the most-derived class, it directly initializes Person.

Structured Pseudocode:

```
CLASS Person
    DECLARE personID, fullName, email
    CONSTRUCTOR(id, name, mail)
        personID <- id
        fullName <- name
        email <- mail
    END CONSTRUCTOR
END CLASS

CLASS Student EXTENDS VIRTUAL Person
    DECLARE gpa, creditsEarned
    CONSTRUCTOR(id, name, mail, grade, credits)
        BASE Person(id, name, mail)
        gpa <- grade
        creditsEarned <- credits
    END CONSTRUCTOR
END CLASS

CLASS Faculty EXTENDS VIRTUAL Person
    DECLARE academicDepartment, rankTitle
    CONSTRUCTOR(id, name, mail, dept, rank)
        BASE Person(id, name, mail)
        academicDepartment <- dept
        rankTitle <- rank
    END CONSTRUCTOR
END CLASS

CLASS TeachingAssistant INHERITS Student, Faculty
    DECLARE stipendAmount, assignedCourse
    CONSTRUCTOR(id, name, mail, grade, credits,
                dept, rank, stipend, course)
        BASE Person(id, name, mail)
        BASE Student(id, name, mail, grade, credits)
```

```
        BASE Faculty(id, name, mail, dept, rank)
        stipendAmount <- stipend
        assignedCourse <- course
    END CONSTRUCTOR

    METHOD DisplayTARecord()
        PRINT personID, fullName, email
        PRINT gpa, creditsEarned
        PRINT academicDepartment, rankTitle
        PRINT stipendAmount, assignedCourse
    END METHOD
END CLASS

MAIN
    DECLARE ta AS TeachingAssistant
    ta <- NEW TeachingAssistant(
        "TA101", "Anu", "anu@univ.edu",
        8.7, 96, "Computer Science",
        "Assistant Professor", 15000, "C++")
    ta.DisplayTARecord()
END MAIN
```

Dry Run / Trace:

Step	Action	Value / Result
1	Initialize virtual Person	TA101, Anu, anu@univ.edu
2	Initialize Student part	GPA=8.7, credits=96
3	Initialize Faculty part	Computer Science, Assistant Professor
4	Initialize TeachingAssistant part	Stipend=15000, Course=C++
5	Display record	All fields accessed unambiguously

Expected Result:

```
Person ID: TA101
Name: Anu
Email: anu@univ.edu
GPA: 8.7
Credits Earned: 96
Department: Computer Science
Rank: Assistant Professor
Stipend: 15000
Assigned Course: C++
```

Question 2: Polymorphic Payroll with Abstract Base Class & Base Pointers

Topic: Abstract Classes, Pure Virtual Functions, Pointers to Derived Classes

Design Overview: Employee is abstract because CalculatePay() is pure virtual. An array of Employee pointers stores addresses of different concrete employee objects. Calling CalculatePay() through each pointer selects the appropriate override.

Structured Pseudocode:

```
ABSTRACT CLASS Employee
   PROTECTED employeeID, name
   CONSTRUCTOR(id, empName)
      employeeID <- id
      name <- empName
   END CONSTRUCTOR
   ABSTRACT METHOD CalculatePay() RETURNS REAL
   METHOD DisplayHeader()
      PRINT employeeID, name
   END METHOD
END CLASS

CLASS SalariedEmployee EXTENDS Employee
   DECLARE monthlySalary
   CONSTRUCTOR(id, name, salary)
      BASE Employee(id, name)
      monthlySalary <- salary
   END CONSTRUCTOR
   METHOD CalculatePay() RETURNS REAL
      RETURN monthlySalary
   END METHOD
END CLASS

CLASS HourlyEmployee EXTENDS Employee
   DECLARE hourlyWage, hoursWorked
   CONSTRUCTOR(id, name, wage, hours)
      BASE Employee(id, name)
      hourlyWage <- wage
      hoursWorked <- hours
   END CONSTRUCTOR
   METHOD CalculatePay() RETURNS REAL
      IF hoursWorked <= 40 THEN
         RETURN hourlyWage * hoursWorked
      ELSE
         RETURN (40 * hourlyWage) +
              ((hoursWorked - 40) * hourlyWage * 1.5)
      END IF
```

```
    END METHOD
END CLASS

MAIN
    DECLARE employees AS ARRAY OF POINTER TO Employee[2]
    employees[0] <- NEW SalariedEmployee("E01", "Ravi", 8500)
    employees[1] <- NEW HourlyEmployee("E02", "Meena", 50, 48)
    total <- 0
    FOR EACH emp IN employees
        emp->DisplayHeader()
        pay <- emp->CalculatePay()
        PRINT pay
        total <- total + pay
    END FOR
    PRINT total
    DELETE employees[0]
    DELETE employees[1]
END MAIN
```

Dry Run / Pay Calculation:

Employee	Calculation	Pay
SalariedEmployee	Monthly salary	8500
HourlyEmployee	(40 × 50) + (8 × 50 × 1.5) = 2000 + 600	2600
Total	8500 + 2600	11100

Expected Result:

```
E01 Ravi: 8500
E02 Meena: 2600
Total disbursement: 11100
```

Memory note: Each dynamically allocated object is deleted exactly once. In a concrete C++ implementation, Employee should have a virtual destructor so deletion through Employee* safely destroys derived objects.

Question 3: Composite Canvas Engine with Member Classes & this Chaining

Topic: Member Classes (Composition), this Pointer Chaining

Design Overview: Circle has a Point2D member representing its center. SetCenter() and SetRadius() return a reference to the current Circle, enabling chained calls.

Structured Pseudocode:

```
CLASS Point2D
```

```
    DECLARE x, y AS REAL
    CONSTRUCTOR(newX = 0.0, newY = 0.0)
        x <- newX
        y <- newY
    END CONSTRUCTOR
    METHOD SetCoordinates(newX, newY)
        x <- newX
        y <- newY
    END METHOD
    METHOD Display()
        PRINT x, y
    END METHOD
END CLASS

CLASS Circle
    DECLARE centerPoint AS Point2D
    DECLARE radius AS REAL
    CONSTRUCTOR()
        centerPoint <- NEW Point2D(0.0, 0.0)
        radius <- 1.0
    END CONSTRUCTOR

    METHOD SetCenter(newX, newY) RETURNS REF Circle
        centerPoint.SetCoordinates(newX, newY)
        RETURN REF THIS
    END METHOD

    METHOD SetRadius(r) RETURNS REF Circle
        IF r > 0 THEN
            radius <- r
        ELSE
            PRINT "Invalid radius"
        END IF
        RETURN REF THIS
    END METHOD

    METHOD Draw()
        PRINT "Center: "
        centerPoint.Display()
        PRINT "Radius: ", radius
    END METHOD
END CLASS

MAIN
    DECLARE myCircle AS Circle
    myCircle.SetCenter(12.5, 30.0).SetRadius(7.5).Draw()
```

```
END MAIN
```

Dry Run / Trace:

Step	Method Call	Center (x,y)	Radius
1	Constructor	(0.0, 0.0)	1.0
2	SetCenter(12.5, 30.0)	(12.5, 30.0)	1.0
3	SetRadius(7.5)	(12.5, 30.0)	7.5
4	Draw()	(12.5, 30.0)	7.5

Expected Output:

```
Center: (12.5, 30.0)
Radius: 7.5
```

Question 4: Multi-Level Autonomous Fleet Telemetry with Base Pointers

Topic: Multilevel Inheritance, Constructors in Derived Classes, Base Pointers

Design Overview: DeliveryDrone derives from AutonomousVehicle, which derives from Vehicle. Constructor arguments are passed from the leaf class to each base class. A Vehicle pointer can inspect base-class specifications; a DeliveryDrone pointer can access full telemetry.

Structured Pseudocode:

```
CLASS Vehicle
    DECLARE vehicleVIN, baseWeightKg
    CONSTRUCTOR(vin, weight)
        vehicleVIN <- vin
        baseWeightKg <- weight
    END CONSTRUCTOR
    METHOD DisplaySpecs()
        PRINT vehicleVIN, baseWeightKg
    END METHOD
END CLASS

CLASS AutonomousVehicle EXTENDS Vehicle
    DECLARE lidarSensorID, firmwareVersion
    CONSTRUCTOR(vin, weight, lidar, firmware)
        BASE Vehicle(vin, weight)
        lidarSensorID <- lidar
        firmwareVersion <- firmware
    END CONSTRUCTOR
END CLASS
```

```
CLASS DeliveryDrone EXTENDS AutonomousVehicle
    DECLARE payloadCapacityKg, batteryChargePercent
    CONSTRUCTOR(vin, weight, lidar, firmware, payload, charge)
        BASE AutonomousVehicle(vin, weight, lidar, firmware)
        payloadCapacityKg <- payload
        batteryChargePercent <- charge
    END CONSTRUCTOR

    METHOD DisplayFlightStatus()
        DisplaySpecs()
        PRINT lidarSensorID, firmwareVersion
        PRINT payloadCapacityKg, batteryChargePercent
    END METHOD
END CLASS

MAIN
    DECLARE dronePtr AS POINTER TO DeliveryDrone
    dronePtr <- NEW DeliveryDrone(
        "VIN-D100", 12.5, "LDR-77",
        "v2.4.1", 5.0, 92)

    DECLARE basePtr AS POINTER TO Vehicle
    basePtr <- dronePtr
    basePtr->DisplaySpecs()

    DECLARE fullPtr AS POINTER TO DeliveryDrone
    fullPtr <- CAST basePtr TO POINTER TO DeliveryDrone
    fullPtr->DisplayFlightStatus()

    DELETE dronePtr
END MAIN
```

Dry Run / Constructor Order:

Order	Constructor	Initialized Data
1	Vehicle	VIN-D100, 12.5 kg
2	AutonomousVehicle	LDR-77, firmware v2.4.1
3	DeliveryDrone	Payload 5.0 kg, battery 92%

Pointer Trace:

Pointer	Static Type	Accessible Members
basePtr	Vehicle*	Vehicle members such as DisplaySpecs()
fullPtr	DeliveryDrone*	Inherited members and

		DisplayFlightStatus()

Expected Result:

```
VIN: VIN-D100
Base weight: 12.5 kg
LiDAR Sensor: LDR-77
Firmware: v2.4.1
Payload capacity: 5.0 kg
Battery charge: 92%
```

Safety note: The pseudocode uses a cast because the program itself created the DeliveryDrone. In production C++, use a checked downcast when the dynamic type is not guaranteed, and use a virtual destructor in the base class if deleting through a base pointer.

Question 5: Dynamic Object Pointer Array & Zero-Copy Sorting Engine

Topic: Pointers to Objects, Arrow Operator (->), Pointer Manipulation

Design Overview: Each Product is dynamically allocated. The catalog array stores Product pointers. Bubble sort compares prices through -> but swaps only pointer values, so the Product objects are not copied or moved.

Structured Pseudocode:

```
CLASS Product
    DECLARE sku AS STRING
    DECLARE price AS REAL
    DECLARE stockCount AS INTEGER
    CONSTRUCTOR(code, cost, stock)
        sku <- code
        price <- cost
        stockCount <- stock
    END CONSTRUCTOR
    METHOD PrintProduct()
        PRINT sku, price, stockCount
    END METHOD
END CLASS

MAIN
    DECLARE N <- 3
    DECLARE catalog AS ARRAY OF POINTER TO Product[N]

    catalog[0] <- NEW Product("Laptop", 1200, 10)
    catalog[1] <- NEW Product("Mouse", 25, 50)
    catalog[2] <- NEW Product("Monitor", 350, 20)

    FOR i <- 0 TO N - 2
```

```
        FOR j <- 0 TO N - i - 2
            IF catalog[j]->price > catalog[j+1]->price THEN
                tempPtr <- catalog[j]
                catalog[j] <- catalog[j+1]
                catalog[j+1] <- tempPtr
            END IF
        END FOR
    END FOR

    FOR i <- 0 TO N - 1
        catalog[i]->PrintProduct()
    END FOR

    FOR i <- 0 TO N - 1
        DELETE catalog[i]
    END FOR
END MAIN
```

Dry Run / Pointer-Swap Trace:

Pass / Comparison	**Compared Prices**	**Action**	**Pointer Order After Action**
Initial	—	—	Laptop → Mouse → Monitor
Pass 1, j=0	1200 > 25	Swap pointers	Mouse → Laptop → Monitor
Pass 1, j=1	1200 > 350	Swap pointers	Mouse → Monitor → Laptop
Pass 2, j=0	25 < 350	No swap	Mouse → Monitor → Laptop

Expected Output (ascending price):

```
Mouse  | Price: 25   | Stock: 50
Monitor| Price: 350  | Stock: 20
Laptop | Price: 1200 | Stock: 10
```

Key Point: Only catalog[j] and catalog[j+1] are exchanged. The dynamically allocated Product objects remain at their original memory addresses; the array changes which pointer refers to each object.